

Chapter 7 - ANTIC and GTIA

ANTIC and GTIA together produce the Atari 8-bit picture model. They are two chips that share the same compositor layer (13) and the same picture; ANTIC fetches the playfield and runs the **display list**, while GTIA chooses colours and handles **player/missile graphics** (the chip's name for hardware sprites). Both chips live in adjacent register regions and are normally programmed together.

7.1 What ANTIC and GTIA can show

Item	Value
Display area	320 × 192 pixels
Border	32 left, 32 right, 24 top, 24 bottom
Total frame	384 × 240 pixels
Colour registers	256-entry palette, register byte = HHHHLLLL
Picture modes	14 (text and bitmap, set per display-list entry)
Sprites	4 8-pixel players + 4 2-pixel missiles
Interrupts	Display-List Interrupt (DLI), Vertical-Blank Interrupt (VBI)

The chip's distinguishing feature is the **display list**: a short program in main memory that tells ANTIC, line by line, what kind of playfield to fetch and what to do at the end of each region. The list is what makes the screen of a typical Atari 8-bit game switch mode and scrolling speed every few scanlines.

7.2 BASIC keywords

The ANTIC and GTIA keywords introduce subcommands for each chip.

Form	Effect
ANTIC ON / ANTIC OFF	Enable / disable the chip.
ANTIC DLIST $addr$	Set the display-list pointer (writes the high and low byte registers).
ANTIC MODE $value$	Write $value$ to DMACTL.
ANTIC SCROLL $hscrol, vscrol$	Write the horizontal and vertical fine-scroll registers (each 0–15).
ANTIC CHBASE hi	Write the character-set base (high byte).
ANTIC PMBASE hi	Write the player-missile base (high byte).
ANTIC NMImask	Write $mask$ to NMIIEN (bit 6 = VBI, bit 7 = DLI).
GTIA COLOR $reg, value$	Write $value$ to colour register reg (0–4 = COLPF0–3, COLBK; 5–8 = COLPM0–3).
GTIA PRIOR $value$	Write $value$ to PRIOR.
GTIA PLAYER $n, x[, size]$	Move player n to horizontal position x ; optionally set its size.
GTIA MISSILE n, x	Move missile n to horizontal position x .
GTIA GRAFP $n, bits$	Write 8-pixel pattern $bits$ to player n .

Form	Effect
GTIA GRAFM <i>bits</i>	Write the 4-missile pattern byte.
GTIA GRAC <i>TLvalue</i>	Write the graphics-control byte.

7.3 The ANTIC register block

ANTIC's registers live at 0xF2100–0xF213F. Every register is a 32-bit word at a 4-byte-aligned address; only the low byte of each is meaningful.

Address	Name	Purpose
0xF2100	ANTIC_DMAC <i>TL</i>	DMA control.
0xF2104	ANTIC_CHAC <i>TL</i>	Character control.
0xF2108	ANTIC_DLIS <i>TL</i>	Display-list pointer, low byte.
0xF210C	ANTIC_DLIS <i>TH</i>	Display-list pointer, high byte.
0xF2110	ANTIC_HSCRO <i>L</i>	Horizontal fine-scroll, 0–15 pixels.
0xF2114	ANTIC_VSCRO <i>L</i>	Vertical fine-scroll, 0–15 lines.
0xF2118	ANTIC_PMBASE	Player-missile base address (high byte).
0xF211C	ANTIC_CHBASE	Character-set base address (high byte).
0xF2120	ANTIC_WSYN <i>C</i>	Write to halt the CPU until horizontal sync.
0xF2124	ANTIC_VCOUN <i>T</i>	Current scanline / 2 (read-only).
0xF2128	ANTIC_PENH	Light-pen X (read-only).
0xF212C	ANTIC_PENV	Light-pen Y (read-only).
0xF2130	ANTIC_NMIEN	NMI enable (bit 6 = VBI, bit 7 = DLI).
0xF2134	ANTIC_NMIS <i>T</i>	NMI status (read); write to clear.
0xF2138	ANTIC_ENABLE	Bit 0 = video enable, bit 1 = PAL timing.
0xF213C	ANTIC_STATU <i>S</i>	Bit 0 = VBlank active (read-only).

7.3.1 DMAC*TL* bits

Bit	Name	Meaning
0–1	Width	00 off, 01 narrow (128 colour clocks), 10 normal (160), 11 wide (192).
2	MISSILE	Enable missile DMA.
3	PLAYER	Enable player DMA.
4	PMRES	0 = double-line P/M, 1 = single-line.
5	DL	Enable display-list DMA. (Required to render anything.)

7.3.2 CHACTL bits

Bit	Name	Meaning
0	BLANK	Force blank in place of inverse video.
1	INVERT	Invert character cells (swap foreground/background).
2	REFLECT	Mirror character rows vertically.

7.4 The display list

ANTIC fetches the playfield by walking a small program in main memory. The program is stored as bytes; each byte is an **instruction**. ANTIC reads the next instruction after every region the previous one has finished.

7.4.1 Instruction encoding

Every instruction is one byte:

bit	7	6	5	4	3	2	1	0
	D	L	V	H	M	M	M	M
					modifiers		---	mode---

The low nibble (mode) is 0–15. The high nibble carries **modifiers** that change how the mode is fetched:

Bit	Modifier	Meaning
4	HSCROL	Enable horizontal fine-scroll for this region.
5	VSCROL	Enable vertical fine-scroll for this region.
6	LMS	The next two bytes hold a 16-bit fetch address; ANTIC reloads its screen pointer from them.
7	DLI	Fire a display-list interrupt at the end of this region.

Two special instructions live in the encoding:

Byte	Name	Effect
0x01	JMP	The next two bytes are an address; jump there.
0x41	JVB	Like JMP , but also wait for the next vertical blank.

7.4.2 The mode list

Modes 2–7 are character modes; modes 8–15 are bitmap modes; mode 0 is one or more blank scanlines.

Mode	Kind	Geometry per row
0	Blank	(opcode >> 4) + 1 blank scanlines (1–8).
2	Text	40 columns, 8 scanlines per row (the standard mode).
3	Text	40 columns, 10 scanlines per row.
4	Text	40 columns, 8 scanlines, multicolour.
5	Text	40 columns, 16 scanlines, multicolour.
6	Text	20 columns, 8 scanlines, 16-pixel-wide characters.

Mode	Kind	Geometry per row
7	Text	20 columns, 16 scanlines.
8	Bitmap	40 pixels wide, 8 scanlines per row.
9	Bitmap	80 pixels wide, 4 scanlines.
10	Bitmap	80 pixels wide, 2 scanlines.
11	Bitmap	160 pixels wide, 1 scanline.
12	Bitmap	160 pixels wide, 1 scanline (alternate colour set).
13	Bitmap	160 pixels wide, 2 scanlines.
14	Bitmap	160 pixels wide, 1 scanline, 4 colours.
15	Bitmap	320 pixels wide, 1 scanline, 2 colours (GTIA modes).

7.4.3 A short example

This eight-byte display list shows one mode-2 region of text at the top of the screen, fires a DLI at its end, then jumps back to its own start and waits for the next frame:

addr	bytes	meaning
\$5000	\$42	mode 2, LMS modifier
\$5001	\$00 \$60	screen RAM starts at \$6000
\$5003	\$82	mode 2 with DLI at the end of the region
\$5004	\$02	another mode-2 region
\$5005	...	(more entries here)
\$50FE	\$41	JVB
\$50FF	\$00 \$50	jump back to \$5000 at next VBlank

Set the display-list pointer with ANTIC_DLSTL/DLISTH and enable display-list DMA with bit 5 of DMACTL.

7.5 Interrupts

ANTIC raises two kinds of interrupt:

- **VBI** (Vertical-Blank Interrupt). Fires once per frame at the start of vertical retrace. Enable with bit 6 of NMIE; status bit 6 of NMIST. Used by the OS for routine per-frame work (timers, joystick scan, colour rotation).
- **DLI** (Display-List Interrupt). Fires at the end of any display-list region whose DLI modifier bit is set. Enable with bit 7 of NMIE; status bit 7 of NMIST. Used for raster effects: change colours, scroll position, or character base mid-screen.

The CPU acknowledges either interrupt by writing any value to NMIST (also known as NMIRE), which clears both pending bits.

7.6 Wait for horizontal sync

Writing any value to ANTIC_WSYNC halts the CPU until the next horizontal-retrace edge. This is the simplest way to time a register change to the start of the next scanline without polling.

7.7 The GTIA register block

GTIA's registers live immediately after ANTIC's, at 0xF2140 through 0xF21FB. Every register is again a 32-bit word at a 4-byte boundary, with only the low byte meaningful.

7.7.1 Colour registers

The playfield uses five colour registers; the four players and four missiles use four more. Players and missiles 0–3 share their colour register (one register per pair).

Address	Name	Used for
0xF2140	GTIA_COLPF0	Playfield colour 0.
0xF2144	GTIA_COLPF1	Playfield colour 1.
0xF2148	GTIA_COLPF2	Playfield colour 2.
0xF214C	GTIA_COLPF3	Playfield colour 3.
0xF2150	GTIA_COLBK	Background and border.
0xF2154	GTIA_COLPM0	Player/missile 0.
0xF2158	GTIA_COLPM1	Player/missile 1.
0xF215C	GTIA_COLPM2	Player/missile 2.
0xF2160	GTIA_COLPM3	Player/missile 3.

Each register holds an 8-bit colour byte in the form HHHHLLLL, where H selects one of 16 hues and L selects one of 16 luminance steps. The full table is in Appendix B.

7.7.2 Control registers

Address	Name	Purpose
0xF2164	GTIA_PRIOR	Priority and GTIA-mode bits.
0xF2168	GTIA_GRCTL	Graphics control.
0xF216C	GTIA_CONSOL	Console switches (read).

PRIOR bits:

Bit	Name	Meaning
0–3	Mix	Player/playfield priority pattern (Atari standard).
4	MULTI	Enable multicolour players.
5	FIFTH	Treat missiles as a single fifth player.
6–7	GTIA mode	Select GTIA-only high-colour modes.

GRCTL bits:

Bit	Name	Meaning
0	MISSILE	Enable missile graphics.
1	PLAYER	Enable player graphics.

Bit	Name	Meaning
2	LATCH	Latch trigger inputs.

7.8 Players and missiles

Each of the four players is 8 pixels wide; each of the four missiles is 2 pixels wide. They can be drawn either from DMA buffers (under ANTIC's control, with DMA_PLAYER/DMA_MISSILE bits set) or by writing directly into the GTIA graphics registers.

Address	Name	Purpose
0xF2170–0xF217C	GTIA_HPOSP0–HPOSP3	Player horizontal position.
0xF2180–0xF218C	GTIA_HPOSM0–HPOSM3	Missile horizontal position.
0xF2190–0xF219C	GTIA_SIZEP0–SIZEP3	Player size. 0 = normal, 1 = double, 3 = quadruple.
0xF21A0	GTIA_SIZEM	All four missile sizes (2 bits each).
0xF21A4–0xF21B0	GTIA_GRAFP0–GRAFP3	Player graphics (8 pixels each).
0xF21B4	GTIA_GRAFM	Missile graphics (2 bits per missile).

7.8.1 Collisions

GTIA reports collisions in a 16-register read-only block at 0xF21B8–0xF21F4. Each register's low nibble is a bit mask of the four object groups (COLPF0–3 for player-vs-playfield, players 0–3 for player-vs-player). The collisions latch from frame to frame until cleared by a write to GTIA_HITCLR (0xF21F8).

7.9 Putting it together

From machine language:

1. Build a display list in main memory.
2. Write its address to ANTIC_DLISTL/DLISTH.
3. Set the picture mode and DMA flags with DMACTL.
4. Load any playfield and player/missile colours into the GTIA colour registers.
5. Write 1 to ANTIC_ENABLE (or ANTIC_ON from BASIC).
6. For raster effects, set the DLI modifier bit in the relevant display-list entries and write a handler that changes colours or scroll values before returning.

The next chapter covers the ULA, the Sinclair-style picture chip on layer 15.